

ECS and Fargate

Containers · Task Definitions · Services · ECR · Auto Scaling · Blue/Green · ShopFlow

AWS Tutorial · ShopFlow

11-01

What is ECS Fargate? Containers Without Servers

■ Amazon ECS with AWS Fargate

ECS (Elastic Container Service) runs Docker containers on AWS. Fargate is the serverless launch type — you define the container and CPU/memory requirements; AWS manages the underlying EC2 instances, OS patching, and cluster scaling. You only manage your containers, not the servers running them.

Example: ShopFlow runs all microservices (product-svc, order-svc, cart-svc, payment-svc) on ECS Fargate. Each service is a Docker container defined in a Task Definition, running as a Service behind an ALB. Fargate scales tasks automatically based on CPU utilization.

Concept	Description	ShopFlow Value
Container	Docker image with your application code + dependencies	shopflow/product-svc:v1.2.3 — Spring Boot JAR + JRE 21
Task Definition	Blueprint: which image, CPU/memory, ports, environment variables, vCPU, GB RAM	product-svc: 1 vCPU, 2 GB RAM, port 8080, reads from S3
Task	One running instance of a Task Definition (like a running Docker container)	product-svc: one Spring Boot process serving HTTP requests
Service	Maintains desired count of tasks, replaces failed ones, integrates with ALB	product-svc: desired=6 tasks; ALB distributes traffic
Cluster	Logical grouping of services and tasks	shopflow-production cluster: all ShopFlow services in one cluster
Fargate	Serverless compute for ECS — no EC2 fleet to manage	AWS provisions capacity; you pay per task vCPU/memory

11-02

ECS Fargate vs Lambda vs EC2 — When to Use Each

The right compute choice depends on workload type, duration, and traffic pattern. ShopFlow uses all three for different purposes.

Dimension	Lambda	ECS Fargate	EC2 + ASG
Request duration	Max 15 minutes	Unlimited	Unlimited
Startup latency	Cold start 200ms-3s (SnapStart)	30-60s (first deploy); 0 for running tasks	Minutes (AMI boot)
Traffic pattern	Spiky/variable; scales to zero	Steady or predictable baseline	High sustained, predictable
Container support	Packaged as ZIP or container image	Native Docker containers	Native Docker via ECS EC2 mode
Server management	Zero — fully serverless	Zero — Fargate handles EC2 fleet	Full — you manage OS, patches, capacity
Max memory	10 GB	120 GB	Depends on instance type (up to 24 TB)
Concurrency model	One invocation per request	Multi-threaded container handles many requests	Same as Fargate
Cost model	Per 100ms execution	Per second vCPU + memory	Per hour instance + storage
ShopFlow usage	Async: notifications, images, cron jobs	Sync APIs: product, order, cart, payment	Not used — Fargate covers all needs

11-03

Create an ECS Cluster — Console Walkthrough

An ECS Cluster is the logical container for all your services and tasks. With Fargate, creating a cluster is just naming it — no EC2 instances to configure.

■ Create ECS Fargate Cluster

1. AWS Console -> search "ECS" -> click Elastic Container Service
2. Left sidebar -> Clusters -> Create cluster
3. Cluster name: shopflow-production
4. Infrastructure: AWS Fargate (serverless) — leave checked
5. Leave EC2 instances unchecked (not needed for Fargate)
6. Monitoring: Enable Container Insights (CloudWatch metrics for all tasks)
7. Tags: Environment=production, Project=shopflow
8. Click "Create" — takes ~30 seconds
9. Verify: cluster status shows "ACTIVE"
10. Note the cluster ARN: arn:aws:ecs:us-east-1:123456789:cluster/shopflow-production

■ aws > ECS > Clusters > shopflow-production

Cluster overview

■ shopflow-production

- Status: ACTIVE | Region: us-east-1 | Infrastructure: Fargate
- Running tasks: 0 | Services: 0 | Registered instances: 0 (Fargate — no EC2)

■ Monitoring

- Container Insights: Enabled (CloudWatch metrics for CPU, memory, network per task)

■ Cluster created successfully. Next: create a Task Definition.

11-04

Task Definitions — Container Blueprint

A Task Definition specifies everything about your container: which image to run, how much CPU and memory it needs, which ports to expose, environment variables, and IAM permissions. Think of it as a Dockerfile for running, not building.

Create Task Definition

Task definition configuration

Task definition family

shopflow-product-svc

Launch type

AWS Fargate ■

Operating system/Architecture

Linux/ARM64 (Graviton — 20% cheaper) ■

Task CPU

1 vCPU ■

Task memory

2 GB ■

Container details

Name

product-svc

Image URI

123456789.dkr.ecr.us-east-1.amazonaws.com/shopflow/product-svc:latest

Container port

8080

Protocol

TCP ■

Environment variables

■ DB_SECRET_ARN = arn:aws:secretsmanager:us-east-1:123:secret:shopflow/aurora

■ REDIS_ENDPOINT = shopflow-redis.cache.amazonaws.com:6379

■ ENVIRONMENT = production

■ Task definition ready to create

Create

11-05

Task Definitions in CDK — Infrastructure as Code

CDK defines Task Definitions as code. This ensures every deployment uses exactly the same configuration and environment variables, with changes tracked in Git.

```
// CDK: ShopFlow product-svc Task Definition FargateTaskDefinition productSvcTaskDef =  
FargateTaskDefinition.Builder .create(this, "ProductSvcTaskDef")  
    .family("shopflow-product-svc") .cpu(1024) // 1 vCPU = 1024 CPU units .memoryLimitMiB(2048) //  
    2 GB RAM .runtimePlatform(RuntimePlatform.builder() .cpuArchitecture(CpuArchitecture.ARM64) //  
    Graviton — 20% cheaper .operatingSystemFamily(OperatingSystemFamily.LINUX) .build())  
    .executionRole(ecsExecutionRole) // pulls image from ECR, writes to CloudWatch Logs
```

```
.taskRole(productSvcTaskRole) // app permissions (read Aurora, write Redis) .build(); // Add
container to task definition ContainerDefinition productSvcContainer =
productSvcTaskDef.addContainer( "product-svc", ContainerDefinitionOptions.builder()
.image(ContainerImage.fromEcrRepository( Repository.fromRepositoryName(this, "ProductSvcRepo",
"shopflow/product-svc"), "v1.2.3")) // always use explicit tag, never "latest" in production
.portMappings(List.of(PortMapping.builder()
.containerPort(8080).protocol(Protocol.TCP).build())) .secrets(Map.of( "DB_PASSWORD",
Secret.fromSecretsManager(auroraSecret, "password"), "REDIS_AUTH",
Secret.fromSecretsManager(redisSecret, "token"))) .environment(Map.of(
"SPRING_PROFILES_ACTIVE", "production", "JAVA_OPTS", "-XX:MaxRAMPercentage=75.0")) // 75% of
2GB = 1.5GB heap .logging(LogDrivers.awsLogs(AwsLogDriverProps.builder()
.streamPrefix("product-svc") .logGroup(new LogGroup(this, "ProductSvcLogs",
LogGroupProps.builder() .retention(RetentionDays.ONE_WEEK).build())) .build()))
.healthCheck(HealthCheck.builder() .command(List.of("CMD-SHELL", "curl -f
http://localhost:8080/actuator/health || exit 1")) .interval(Duration.seconds(30))
.timeout(Duration.seconds(5)) .retries(3) .build()) .build());
```

11-06

ECS Service — Maintain Running Tasks

An ECS Service runs and maintains a specified number of Task instances. If a task fails, the Service automatically starts a replacement. The Service integrates with the ALB to register/deregister tasks as health checks pass/fail.

```
// CDK: ECS Fargate Service + ALB integration ApplicationLoadBalancer alb =
ApplicationLoadBalancer.Builder .create(this, "ShopFlowALB") .vpc(vpc) .internetFacing(true)
.build(); ApplicationTargetGroup productSvcTargetGroup = ApplicationTargetGroup.Builder
.create(this, "ProductSvcTG") .vpc(vpc) .port(8080) .protocol(ApplicationProtocol.HTTP)
.targetType(TargetType.IP) // Fargate tasks registered by IP, not instance ID
.healthCheck(HealthCheckProps.builder() .path("/actuator/health") .healthyHttpCodes("200")
.interval(Duration.seconds(30)) .unhealthyThresholdCount(3) .build())
.deregistrationDelay(Duration.seconds(30)) // wait 30s before removing from ALB .build();
FargateService productSvcService = FargateService.Builder .create(this, "ProductSvcService")
.cluster(shopflowCluster) .taskDefinition(productSvcTaskDef) .desiredCount(3) // 3 tasks across
3 AZs (1 per AZ) .vpcSubnets(SubnetSelection.builder()
.subnetType(SubnetType.PRIVATE_WITH_EGRESS) // app-tier subnets .build())
.securityGroups(List.of(productSvcSg)) .serviceConnectConfiguration(...) // optional:
service-to-service discovery .deploymentController(DeploymentController.builder()
.type(DeploymentControllerType.ECS) // rolling update (not blue/green) .build())
.circuitBreaker(DeploymentCircuitBreaker.builder() .rollback(true) // auto-rollback if
deployment fails health checks .build()) .build(); // Register tasks with ALB target group
productSvcService.attachToApplicationTargetGroup(productSvcTargetGroup);
```

11-07

ECS Service Auto Scaling

ECS Service Auto Scaling adds or removes tasks based on CloudWatch metrics. ShopFlow uses CPU-based scaling as the primary policy and request-count scaling as a secondary policy — whichever requires more tasks wins.

```
// CDK: ECS Auto Scaling ScalableTaskCount scalableTarget =
productSvcService.autoScaleTaskCount( EnableScalingProps.builder() .minCapacity(3) // minimum 3
tasks (1 per AZ for HA) .maxCapacity(30) // maximum 30 tasks on Black Friday .build()); //
Policy 1: Scale on CPU utilization scalableTarget.scaleOnCpuUtilization("CpuScaling",
CpuUtilizationScalingProps.builder() .targetUtilizationPercent(60) // keep CPU at 60%
.scaleInCooldown(Duration.minutes(3)) // wait 3min before scaling in
.scaleOutCooldown(Duration.seconds(60)) // scale out quickly (1 min) .build()); // Policy 2:
Scale on ALB request count per target
scalableTarget.scaleOnRequestCountPerTarget("RequestScaling",
RequestCountScalingProps.builder() .requestsPerTarget(1000) // 1,000 req/min per task =
comfortable load .targetGroup(productSvcTargetGroup) .scaleInCooldown(Duration.minutes(3))
.scaleOutCooldown(Duration.seconds(60)) .build()); // Policy 3: Scheduled scaling for Black
Friday scalableTarget.scaleOnSchedule("BlackFridayScale", ScalingSchedule.builder()
.schedule(Schedule.cron(CronOptions.builder()
.hour("15").minute("0").month("11").day("29").build())) .minCapacity(15) // pre-scale to 15
tasks at 3 PM (before 4 PM rush) .build());
```

11-08

ECR — Container Image Registry

Amazon ECR (Elastic Container Registry) stores your Docker images. ECS Fargate pulls images from ECR during task launch. ECR is integrated with IAM, encrypted at rest, and scanned for vulnerabilities.

■ Push Image to ECR

1. ECR Console -> Create repository

2. Repository name: shopflow/product-svc

3. Image scanning: Enable "Scan on push" (finds CVEs automatically)

4. Encryption: AWS managed key

5. Click Create

6. Click "View push commands" -> follow the 4 commands shown:

7. Step 1: `aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin .dkr.ecr.us-east-1.amazonaws.com`

8. Step 2: `docker build -t shopflow/product-svc .`

9. Step 3: `docker tag shopflow/product-svc:latest .dkr.ecr.us-east-1.amazonaws.com/shopflow/product-svc:v1.2.3`

10. Step 4: `docker push .dkr.ecr.us-east-1.amazonaws.com/shopflow/product-svc:v1.2.3`

11. Verify: ECR Console -> repository -> see image with tag v1.2.3 and size

ECR Feature	Configuration	ShopFlow Setting
Lifecycle policy	Auto-delete old images to control storage costs	Keep latest 10 tagged images; delete untagged images after 1 d
Image scanning	Scan each push for known CVEs in OS packages and dependencies	Scans enabled; block CI/CD pipeline if CRITICAL CVE fo
Cross-region replication	Replicate images to additional regions automatically	Replicate to us-west-2 for DR; ECS Fargate in DR pulls locally
Pull-through cache	Cache public Docker Hub / ECR Public images in private ECR	Cache enabled; use private ECR

11-09

Deploying Updates — Rolling Update and Blue/Green

ECS supports two deployment strategies. Rolling update replaces tasks gradually — some old, some new tasks run simultaneously during deployment. Blue/Green (via CodeDeploy) runs a complete new environment in parallel and cuts over with a traffic shift.

Strategy	How It Works	Pros	Cons	ShopFlow Usage
Rolling update (ECS)	Replace % of tasks with new version	On ALB health checks; gradual	Requires manual intervention	Can be used for non-critical updates
Blue/Green (CodeDeploy)	Full set of new tasks (green), shift ALB traffic	Shift ALB traffic (Shift 25% at a time)	Requires manual intervention	Can be used for critical updates

INFO Deployment Circuit Breaker

With Rolling Update + circuit breaker enabled (CDK: `.circuitBreaker(rollback=true)`), ECS monitors task health during deployment. If the new task version fails its health check 3 times, ECS automatically rolls back to the previous task definition version. No manual intervention needed. This is the safety net that makes rolling deploys safe for ShopFlow.

11-10

Service-to-Service Communication and Discovery

ShopFlow microservices need to call each other: order-svc calls product-svc (for price validation), payment-svc calls order-svc (to update order status). ECS Service Connect provides built-in service discovery and load balancing without an API Gateway or service mesh.

```
// CDK: ECS Service Connect (service-to-service discovery) // Define a namespace (DNS domain for
internal services) Namespace shopflowNamespace = Namespace.Builder .create(this,
"ShopFlowNamespace") .namespaceName("shopflow.internal") .vpc(vpc) .build(); // product-svc:
registers as product-svc.shopflow.internal FargateService productSvcService =
FargateService.Builder.create(...) .serviceConnectConfiguration(ServiceConnectProps.builder()
.namespace("shopflow.internal") .services(List.of(ServiceConnectService.builder()
.portMappingName("http") // matches portMappings name in task def .port(8080)
.dnsName("product-svc") // DNS: product-svc.shopflow.internal .build())) .build()) .build(); //
order-svc calls product-svc internally by DNS name // No ALB, no API Gateway, no hardcoded IPs:
RestTemplate restTemplate = new RestTemplate(); Product product = restTemplate.getForObject(
"http://product-svc.shopflow.internal:8080/products/" + productId, Product.class); // Service
Connect provides: client-side load balancing, retries, // circuit breaking, and metrics per
service pair – no Envoy proxy to manage
```

11-11

Container Logging and Observability

ECS Fargate containers send logs to CloudWatch Logs via the awslogs driver. With Container Insights enabled on the cluster, you get automatic metrics for CPU, memory, network, and storage per task and service.

■ aws > CloudWatch > Container Insights > shopflow-production

ECS Container Insights

■ Services — last 5 minutes

■ product-svc: CPU 45% | Memory 68% | Tasks running: 6 | Network in: 42 MB/s

■ order-svc: CPU 22% | Memory 51% | Tasks running: 3 | Network in: 18 MB/s

■ cart-svc: CPU 31% | Memory 43% | Tasks running: 3 | Network in: 8 MB/s

■ payment-svc: CPU 18% | Memory 35% | Tasks running: 3 | Network in: 5 MB/s

■ Alarms

■ OK: product-svc CPU < 80%

■ OK: order-svc Memory < 85%

■ Cart-svc CPU trending up (31%): approaching scale-out threshold of 60%

Metric	Alarm Threshold	Action
Service CPU utilization	> 80% for 5 minutes	Auto scaling triggers at 60%; at 80% investigate if scaling is k
Service Memory utilization	> 85% for 5 minutes	Container OOM kills at ~100%; alert at 85% to add memory o
Task count below desired	< desiredCount for 3 minutes	Tasks crashing; check container logs for startup errors or heal
ALB 5xx error rate	> 1% for 2 minutes	Application errors; check CloudWatch Logs Insights for excep
ALB target response time P99	> 1,000ms for 5 minutes	Slow responses; check CPU/memory; check downstream Aur

11-12

ECS Task IAM Roles — Least Privilege

ECS Fargate uses two IAM roles: the Execution Role (used by ECS to pull the image and send logs) and the Task Role (used by your application code to call AWS services). Never use the same role for both, and never use overly broad policies.

```
// CDK: Two separate IAM roles for ECS tasks // 1. Execution Role – used by ECS infrastructure
(not your app code) Role executionRole = Role.Builder.create(this, "EcsExecutionRole")
.roleName("shopflow-ecs-execution-role") .assumedBy(new
ServicePrincipal("ecs-tasks.amazonaws.com")) .managedPolicies(List.of( // Required: pull images
```

```
from ECR, write logs to CloudWatch ManagedPolicy.fromAwsManagedPolicyName(
"service-role/AmazonECSTaskExecutionRolePolicy"))) .build(); // Grant: read Secrets Manager
secrets (for env var injection) auroraSecret.grantRead(executionRole);
redisSecret.grantRead(executionRole); // 2. Task Role – used by your application code (Spring
Boot) Role taskRole = Role.Builder.create(this, "ProductSvcTaskRole")
.roleName("shopflow-product-svc-task-role") .assumedBy(new
ServicePrincipal("ecs-tasks.amazonaws.com")) .build(); // Grant only what product-svc needs:
productsTable.grantReadData(taskRole); // read DynamoDB products
productsBucket.grantRead(taskRole); // read S3 product images
openSearchDomain.grantReadWrite(taskRole); // search index queries // Do NOT grant: Aurora
access (product-svc uses read replica via JDBC in Secrets Manager) // Do NOT grant: broad s3:*
or dynamodb:* – scope to specific resources
```

11-13

Fargate Spot — 70% Cost Savings

Fargate Spot uses spare AWS capacity at up to 70% discount versus standard Fargate. AWS can reclaim Spot capacity with a 2-minute warning. Use Spot for fault-tolerant workloads; use standard Fargate for critical tasks that cannot be interrupted.

Service	Standard Fargate	Fargate Spot	Recommendation
product-svc (read API)	3 tasks minimum	3 additional tasks (6 total)	50% Spot safe: lose a Spot task, 5 remain; ALB re-route
order-svc (checkout)	3 tasks (all Standard)	0 Spot tasks	All Standard: mid-checkout interruption = lost order
payment-svc (payment)	3 tasks (all Standard)	0 Spot tasks	All Standard: payment interruption = potential double-charge
image-processor (batch)	0 Standard	10 tasks (all Spot)	All Spot safe: stateless image resize; retry on interruption
daily-report (cron)	0 Standard	1 task (Spot)	All Spot safe: if interrupted, EventBridge retries Lambda

```
// CDK: Capacity Provider Strategy (mix Standard + Spot) FargateService productSvcService =
FargateService.Builder.create(this, "ProductSvc") .cluster(shopflowCluster)
.taskDefinition(productSvcTaskDef) .desiredCount(6) .capacityProviderStrategies(List.of(
CapacityProviderStrategy.builder() .capacityProvider("FARGATE") // Standard Fargate
.weight(1).base(3) // base=3: always run 3 Standard tasks .build(),
CapacityProviderStrategy.builder() .capacityProvider("FARGATE_SPOT") // Spot (up to 70%
cheaper) .weight(3) // scale-out prefers Spot .build())) .build();
```

11-14

Hands-On Lab — Deploy ShopFlow Product Service

Deploy a containerized Spring Boot service on ECS Fargate. You will build the image, push to ECR, create a Task Definition, run it as a Service, and verify via ALB. Time estimate: 60 minutes.

■ Lab: Deploy product-svc on ECS Fargate

1. Create ECR repository: shopflow/product-svc-lab
2. Build Docker image: `docker build -t shopflow/product-svc-lab .` (use sample Dockerfile)
3. Tag and push to ECR (follow ECR "View push commands")
4. Create ECS Cluster: shopflow-lab (Fargate only)
5. Create Task Definition: product-svc-lab (Fargate, Linux/ARM64, 0.25 vCPU, 512MB)
6. Container: image=your ECR URI, port=8080, log driver=awslogs
7. Create VPC ALB: shopflow-lab-alb (internet-facing, port 80)
8. Create Target Group: shopflow-product-svc-lab (IP type, port 8080, health=/health)
9. Create ECS Service: product-svc-lab, desired=2 tasks, attach to ALB target group
10. Wait for tasks to reach RUNNING status (~60 seconds)
11. Test: `curl http://shopflow-lab-alb-.us-east-1.elb.amazonaws.com/products`
12. Scale: ECS Service -> Update -> desired count=4, verify 2 new tasks start
13. Cleanup: delete Service (scale to 0 first), Task Definition, Cluster, ALB, ECR repo

11-15

ShopFlow ECS Fargate — Complete Architecture

The complete ShopFlow ECS architecture: all microservices in one cluster, traffic from the internet through CloudFront -> WAF -> ALB -> ECS tasks in private subnets.

ShopFlow ECS Fargate Architecture (us-east-1)

Internet Traffic Flow

CloudFront CDN -> WAF (DDoS + bot protection)

WAF -> ALB (us-east-1a/1b/1c, internet-facing)

shopflow-production ECS Cluster

product-svc: 6 tasks (2/AZ), 1vCPU/2GB, Fargate+Spot

order-svc: 3 tasks (1/AZ), 2vCPU/4GB, Fargate only

cart-svc: 3 tasks (1/AZ), 0.5vCPU/1GB, Fargate+Spot

payment-svc: 3 tasks (1/AZ), 1vCPU/2GB, Fargate only

Data Layer (private subnets)

Aurora PostgreSQL 15 (Multi-AZ, 3 read replicas)

ElastiCache Redis 7 (Cluster mode, 3 shards x 2 nodes)

DynamoDB (cart, sessions, idempotency tables)

Service	Tasks	vCPU	Memory	ALB Path	Health Check
product-svc	6 (min 3)	1.0	2 GB	/products/*	/actuator/health
order-svc	3 (min 3)	2.0	4 GB	/orders/*	/actuator/health
cart-svc	3 (min 3)	0.5	1 GB	/cart/*	/actuator/health
payment-svc	3 (min 3)	1.0	2 GB	/payments/*	/actuator/health

11-16

Common Questions and Best Practices

Question	Answer
How do I run database migrations safely with ECS?	Use a custom ECS task (not Service) triggered by CodePipeline before deploying the new Service version.
How do I share files between ECS tasks?	Use EFS (Elastic File System) as a shared volume mounted into multiple tasks. For ShopFlow: no need.
Why are my tasks stopping with exit code 137?	Exit code 137 = OOM kill (Out of Memory). Container tried to allocate more memory than Task Definition memory.
How long does a Fargate task take to start?	60-90 seconds for first deploy: ECR image pull (~20s for 200MB image) + container start (~5s) + health check.

Best Practice	Anti-Pattern to Avoid
Use explicit image tags (v1.2.3) in Task Definitions; never use latest.	Using latest in ECS task definition. When to pull new image; stale images deployed.
Enable deployment circuit breaker with rollback=true	No circuit breaker — bad deployment stays running, serving errors until manually stopped.
Use ARM64 (Graviton) for all Fargate tasks — 20% cheaper.	Defaulting to x86 for all ShopFlow tasks are CPU-bound Spring Boot; arm64 is better.
Health check endpoint returns 200 only when app is fully ready (DB always ready).	Health check always returns 200 — ALB sends traffic to task before Spring Boot is ready.
Set JAVA_OPTS=-XX:MaxRAMPercentage=75.0 to bound JVM heap limit.	No JVM heap limit. If tasks use 80-90% of host memory, OOM kills task.

11-17

ECS Exec — Debug Running Fargate Tasks

ECS Exec lets you run commands inside a running Fargate task without needing SSH or a bastion host. Essential for debugging production issues.

■ Enable and Use ECS Exec

1. Enable on Service: ECS Console -> Service -> Update -> Enable execute command
2. Task Role must have: ssmmessages:CreateControlChannel, CreateDataChannel, OpenControlChannel, OpenDataChannel
3. Verify tasks: `aws ecs list-tasks --cluster shopflow-production --service-name product-svc`
4. Connect: `aws ecs execute-command --cluster shopflow-production --task --container product-svc --interactive --command "/bin/sh"`
5. Inside container: `curl http://localhost:8080/actuator/health`
6. Check env vars: `env | grep DB`
7. Test Redis connectivity: `nc -zv shopflow-redis.cache.amazonaws.com 6379`
8. Exit: type "exit" (container keeps running)

INFO ECS Exec vs SSH vs Bastion

Traditional: bastion host -> SSH into EC2 -> docker exec. ECS Exec: direct SSM Session Manager tunnel from your laptop into the Fargate container — no bastion, no SSH keys, no EC2. All sessions logged to CloudTrail. Disable ECS Exec after debugging to reduce attack surface.

ECS integrates with Secrets Manager to inject secrets at task startup as environment variables, without your application code ever handling secret retrieval directly.

```
// CDK: inject Secrets Manager values as container env vars
ContainerDefinitionOptions.builder().secrets(Map.of( "DB_PASSWORD",
EcsSecret.fromSecretsManager(auroraSecret, "password"), "REDIS_AUTH_TOKEN",
EcsSecret.fromSecretsManager(redisSecret, "token"), "FEATURE_FLAGS",
EcsSecret.fromSsmParameter(featureFlagParam))) .environment(Map.of( "SPRING_PROFILES_ACTIVE",
"production", "SERVER_PORT", "8080")) .build(); // In Spring Boot – just read env var as normal:
// @Value("${DB_PASSWORD}") private String dbPassword; // ECS decrypts and injects before
Spring context starts
```

Secret Store	Cost	ShopFlow Examples
Secrets Manager	\$0.40/secret/month	Aurora password, Redis auth token, Stripe API key
SSM Parameter Store (Standard)	Free up to 10,000 params	Redis endpoint, S3 bucket names, feature flags

Fargate charges per second for vCPU and memory while tasks are running. ARM64 (Graviton) saves 20% vs x86 at the same performance.

Service	Tasks	vCPU	Memory	Monthly (Standard)	Monthly (w/Spot)
product-svc	6 avg	1.0	2 GB	\$166	\$83 (50% Spot)
order-svc	3 avg	2.0	4 GB	\$166	\$166 (all Standard)
cart-svc	3 avg	0.5	1 GB	\$28	\$14 (50% Spot)
payment-svc	3 avg	1.0	2 GB	\$83	\$83 (all Standard)
TOTAL	15 tasks	—	—	\$443/month	\$346/month (-22%)

TIP ARM64 + Spot = 38% Savings

ARM64 (Graviton2) Fargate costs 20% less than x86 for equal or better performance on Java Spring Boot. Adding Fargate Spot for fault-tolerant services brings total savings to ~38% vs all-Standard x86. ShopFlow saves ~\$170/month on base task costs.

Black Friday generates 10x normal traffic. ECS auto-scaling handles demand, but the 60-second task startup lag means you must pre-scale before the traffic spike hits.

Checklist Item	Action
Scheduled pre-scaling	Scale to min 10 tasks per service at 3 PM (before 4 PM rush): EventBridge Rule -> ECS UpdateService
Max task count increase	Verify maxCapacity=30; request Fargate capacity increase in all 3 AZs via Service Quotas 2 weeks
Deployment freeze	No deployments Nov 27-Dec 1 unless P0 fix; code freeze enforced via CodePipeline approval gate
Load test	Artillery at 3x expected peak 1 week before; verify P99 < 1s, 5xx < 0.1%, tasks scale correctly
Spot reclamation plan	3 Standard base tasks always running per service; Spot adds capacity above base only
ALB draining	30s deregistration delay ensures in-flight requests complete before scale-in removes a task

A Fargate Task can run multiple containers sharing the same network (localhost) and storage volumes. The sidecar pattern adds helper containers alongside your app: X-Ray daemon, log forwarder, or Envoy proxy.

```
// CDK: Main app + X-Ray daemon sidecar FargateTaskDefinition taskDef =
FargateTaskDefinition.Builder.create(this,
"ProductSvcTask").cpu(1024).memoryLimitMiB(2048).build(); taskDef.addContainer("product-svc",
ContainerDefinitionOptions.builder().image(ContainerImage.fromEcrRepository(repo, "v1.2.3"))
.portMappings(List.of(PortMapping.builder().containerPort(8080).build())) .essential(true) //
task stops if this container stops .build()); // X-Ray sidecar: receives traces on
localhost:2000 UDP taskDef.addContainer("xray-daemon", ContainerDefinitionOptions.builder()
.image(ContainerImage.fromRegistry("public.ecr.aws/xray/aws-xray-daemon:latest"))
.cpu(32).memoryLimitMiB(256) .portMappings(List.of(PortMapping.builder()
.containerPort(2000).protocol(Protocol.UDP).build())) .essential(false) // app keeps running if
xray-daemon crashes .build());
```

11-22

ECS Service Connect — Internal Service Discovery

ECS Service Connect provides built-in service discovery for ECS tasks to call each other by name. order-svc calls product-svc as `http://product-svc.shopflow.internal:8080` — no ALB, no hardcoded IPs needed.

```
// CDK: Service Connect configuration FargateService productSvcService =
FargateService.Builder.create(...) .serviceConnectConfiguration(ServiceConnectProps.builder()
.namespace("shopflow.internal") .services(List.of(ServiceConnectService.builder()
.portMappingName("http") .port(8080) .dnsName("product-svc") // resolves to
product-svc.shopflow.internal .build())) .build()) .build(); // order-svc calls product-svc by
DNS name — no service registry needed Product product = restTemplate.getForObject(
"http://product-svc.shopflow.internal:8080/products/" + productId, Product.class); // Service
Connect provides: client-side load balancing, retries, // circuit breaking, and per-service
metrics in CloudWatch
```

Feature	Service Connect	ALB for Internal
Cost	Included in Fargate (no extra charge)	ALB: ~\$22/month minimum per ALB
Service discovery	DNS auto-updated as tasks start/stop	Manual Target Group registration
Observability	Built-in per-service metrics and traces	ALB access logs (separate setup)
Best for	ECS task-to-task communication	External (internet) or cross-VPC traffic

11-23

Fargate for Batch Jobs — Run-to-Completion Tasks

ECS Tasks (not Services) run to completion and exit. ShopFlow uses this for nightly jobs: generating PDF reports, reprocessing failed orders, and bulk image optimization. AWS Batch extends this with job queues and compute environments.

Job Type	ECS Task (manual/scheduled)	AWS Batch on Fargate
Trigger	EventBridge Rule, API call, CodePipeline	AWS Batch job submission or scheduled
Concurrency	Manual: run N tasks simultaneously	Automatic: job queue manages concurrent jobs
Job dependencies	Not built-in	Native: Job B starts after Job A succeeds
Retry policy	ECS restart policy (limited)	Per-job retry with backoff
ShopFlow usage	Daily report (1 task), DB migration (1 task)	Bulk image resize campaign (100+ tasks)

```
// Run a one-time ECS Task (e.g., DB migration before deployment)
ecsClient.runTask(RunTaskRequest.builder().cluster("shopflow-production")
.launchType(LaunchType.FARGATE) .taskDefinition("shopflow-db-migration:5") // specific revision
.networkConfiguration(NetworkConfiguration.builder()
.awsVpcConfiguration(AwsVpcConfiguration.builder().subnets(List.of("subnet-app-1a",
"subnet-app-1b")) .securityGroups(List.of("sg-migrations")))
.assignPublicIp(AssignPublicIp.DISABLED) .build()) .build()) .build()); // Poll until task
stops, then check exit code // exitCode=0: migration succeeded; proceed with service deployment
// exitCode=1: migration failed; abort deployment; rollback
```

ECS uses the container health check to determine if a task is healthy. If a task fails health checks, the ECS Service stops it and starts a replacement. The health check must be carefully designed: too strict causes false restarts; too lenient hides real failures.

Health Check Phase	ECS Behavior	ShopFlow Config
startPeriod (grace period)	ECS ignores health check failures for this duration after a task starts. During this time, the task is in the PENDING state.	<code>startPeriod=60s</code> (Spring Boot: <code>spring.docker.startPeriod=60s</code>)
interval	How often to run the health check. Too frequent = excessive CPU usage; too infrequent = slow detection of failures.	<code>interval=30s</code> (Spring Boot: <code>spring.docker.healthCheckInterval=30s</code>)
timeout	How long to wait for health check response. Spring Boot uses <code>spring.docker.healthCheckTimeout=5s</code> by default.	<code>timeout=5s</code> (Spring Boot: <code>spring.docker.healthCheckTimeout=5s</code>)
unhealthyThreshold (retries)	Number of consecutive failures before marking the task as UNHEALTHY . ECS then replaces the task.	<code>unhealthyThreshold=3</code> (Spring Boot: <code>spring.docker.unhealthyThreshold=3</code>)

INFO /actuator/health for Spring Boot

Spring Boot Actuator `/actuator/health` endpoint checks all health indicators: DataSource (Aurora connection pool), Redis, disk space, and any custom checks. Returns 200 OK only when ALL components are healthy. This is the ideal ECS health check endpoint — if Aurora is unreachable, the health check fails and ECS replaces the task, preventing the ALB from routing traffic to a task that cannot serve requests.

ECS rolling deployments replace old tasks with new ones gradually. The ALB deregisters old tasks only after new ones pass health checks. With correct settings, users experience zero downtime during deployments.

Setting	Value	Purpose
minimumHealthyPercent	100%	ECS must maintain 100% of desired tasks during deployment — no capacity reduction.
maximumPercent	200%	ECS can run up to 200% of desired count during deployment — with desired tasks, it can launch up to 100% more.
deploymentCircuitBreaker	enabled + rollback=true	If > 50% of new tasks fail health checks, ECS automatically reverts to previous version.
deregistrationDelay	30 seconds (ALB setting)	ALB waits 30s after marking task draining before stopping it — in-flight requests complete.
healthCheckGracePeriod	60 seconds (ECS Service Setting)	ECS waits 60s before checking ALB health for newly started tasks — gives time for startup.

Container Insights provides automatic CloudWatch metrics for every ECS cluster, service, and task — CPU, memory, network, and storage. Enable it at cluster creation and you get dashboards immediately.

Metric	Unit	Alarm Threshold	ShopFlow Action
CPUUtilization (Service)	Percent	> 80% for 5 min	Auto scaling triggers at 60%; 80% alarm means scaling is needed.
MemoryUtilization (Service)	Percent	> 85% for 5 min	OOM risk; increase task memory or fix memory leak; check logs.
RunningTaskCount	Count	< desiredCount for 3 min	Tasks crashing; check CloudWatch Logs for startup errors.
DeploymentRollback (EventBridge Event)	Count	Any rollback event	Circuit breaker triggered; new deployment failed; investigate.
ALB RequestCount 5xx	Count	> 1% error rate for 2 min	Application errors; query CloudWatch Logs Insights for errors.
ALB TargetResponseTime P99	Seconds	> 1.0s for 5 min	Slow responses; correlate with CPU/memory or downstream services.

Capacity Providers control which compute (Standard Fargate or Fargate Spot) runs each service, and in what proportion. The strategy below gives each service 3 guaranteed Standard tasks (base) plus additional Spot tasks as

traffic grows.

```
// CDK: Mixed capacity provider strategy FargateService productSvc =
FargateService.Builder.create(this, "ProductSvc") .cluster(shopflowCluster)
.taskDefinition(productSvcTaskDef) .desiredCount(6) .capacityProviderStrategies(List.of(
CapacityProviderStrategy.builder() .capacityProvider("FARGATE") // Standard Fargate .weight(1)
.base(3) // ALWAYS run at least 3 Standard tasks (HA minimum) .build(),
CapacityProviderStrategy.builder() .capacityProvider("FARGATE_SPOT") // Spot (up to 70%
cheaper) .weight(4) // 4x preference for Spot when scaling out .build())) .build(); // With
desiredCount=6: // base=3 Standard tasks always running (cannot be preempted) // 3 additional
tasks use Spot (weight=4 means Spot strongly preferred) // If Spot is reclaimed: ECS replaces
with new Spot (or Standard if Spot unavailable) // Total cost: 3 Standard + 3 Spot = ~$83 + ~$25
= $108/month vs $166 all Standard
```

Service	base (Standard)	Spot tasks	Rationale
product-svc	3 tasks	3 tasks (50% Spot)	Read API: stateless; Spot loss reduces capacity but 3 Stan
order-svc	3 tasks	0 (no Spot)	Mid-checkout Spot reclaim = lost order; all Standard for tran
cart-svc	3 tasks	3 tasks (50% Spot)	Cart operations are idempotent; Spot loss is safe; retried au
payment-svc	3 tasks	0 (no Spot)	Payment operations must not be interrupted; all Standard

11-28

Fargate Networking — awsvpc Mode and ENIs

Every Fargate task gets its own Elastic Network Interface (ENI) with a private IP address in your VPC. This is the awsvpc networking mode — the only mode available for Fargate. Understanding it helps with security group design and IP address planning.

Networking Aspect	Details	ShopFlow Config
Network mode	awsvpc only for Fargate — each task gets its own ENI with private IP for 10.0.4.0/24, 10.0.5.0/24, 10.0.6.0/24	taskEniPrivateIPPrefix(10.0.4.0/24)
IP addresses consumed	1 private IP per running task. 6 product-svc tasks x 622s subnets = 3732 private IPs	Each 622s subnet is 256 IP size for Peak task 30 tasks x 4 services = 7680
Security groups	Attach to the task (ENI), not the EC2 instance. One SG per service type (8000 doors At S3, Greengrass, etc, pay 54c)	One SG per service type (8000 doors At S3, Greengrass, etc, pay 54c)
Public IP	Not recommended. Fargate tasks in private subnets. Assign Public IP = Gateway for all ShopFlow tasks; NAT Gateway	Assign Public IP = Gateway for all ShopFlow tasks; NAT Gateway
DNS resolution	ECS Service Connect uses Cloud Map internal DNS. All ECS tasks use the same DNS. FargateType (FargateType.IP) private IPs or	Assign Public IP = Gateway for all ShopFlow tasks; NAT Gateway

11-29

ECS Anywhere — Run ECS Tasks On-Premises

ECS Anywhere extends ECS to run tasks on your own servers (on-premises data center, edge servers, or servers at other cloud providers) while managing them from the same ECS Console and API. Not used by ShopFlow, but relevant for enterprises with hybrid cloud requirements.

Feature	ECS (Fargate)	ECS Anywhere
Infrastructure	AWS-managed Fargate compute	Your own servers: bare metal, VMs, or EC2 in another region
Registration	Automatic (Fargate is serverless)	Install ECS agent + SSM agent on each server; register with cluster
Networking	awsvpc (VPC-native)	bridge or host network mode (no VPC)
AWS service access	Native (IAM, Secrets Manager, ECR via VPC endpoints)	Overcome DNS to AWS APIs over internet or Direct Connect
Cost	Fargate per-second pricing	\$0.01025/hour per registered vCPU + \$0.00112/hour per GB RAM
Use case	Most workloads	Regulatory requirements (data must stay on-premises), low-latency

INFO ShopFlow Hybrid Use Case

ShopFlow does not use ECS Anywhere today. A future use case: POS (point-of-sale) terminals in ShopFlow physical stores run ECS tasks locally for offline order processing, syncing to AWS when connectivity is available. ECS Anywhere would manage these edge devices with the same CDK + CodePipeline pipeline used for cloud tasks.

11-30

ECS Quick Reference — Commands and Key Settings

Essential AWS CLI commands for day-to-day ECS operations. Run these from your laptop with appropriate IAM permissions or from within the AWS Console CloudShell.

Operation	AWS CLI Command
List running tasks	<code>aws ecs list-tasks --cluster shopflow-production --service-name product-svc</code>
Describe task (get IP, status)	<code>aws ecs describe-tasks --cluster shopflow-production --tasks <task-arn></code>
Force new deployment (rolling)	<code>aws ecs update-service --cluster shopflow-production --service product-svc --force-new-deployment</code>
Scale service	<code>aws ecs update-service --cluster shopflow-production --service product-svc --desired-count 10</code>
Stop a task (ECS replaces it)	<code>aws ecs stop-task --cluster shopflow-production --task <task-arn> --reason "debugging"</code>
ECS Exec into container	<code>aws ecs execute-command --cluster shopflow-production --task <arn> --container product-svc --interactive</code>
View service events	<code>aws ecs describe-services --cluster shopflow-production --services product-svc --query "services[0].events"</code>
Check deployment status	<code>aws ecs describe-services --cluster shopflow-production --services product-svc --query "services[0].deployments"</code>

11-31

Common Questions — ECS Fargate

Question	Answer
My task keeps stopping with exit code 137. What is 137?	Exit 137 (SIGKILL) is common. Your container exceeded the Task Definition memory limit and Linux killed it.
How do I run database migrations safely with ECS?	Use <code>aws ecs run-task</code> (run-task API call, not a Service) triggered by CodePipeline before deploying the app.
My tasks pass health checks but ALB still can't connect to the target. Causes: (1) Security group on ECS task does not allow inbound traffic from ALB. (2) Target group health check path is wrong.	
Can I use Docker Compose files with ECS?	AWS Copilot CLI and Amazon ECS Compose X can convert Docker Compose files to ECS Task Definitions.

11-32

Well-Architected Review — ShopFlow ECS Fargate

The AWS Well-Architected Framework has five pillars. Here is how ShopFlow ECS implementation scores against each pillar, with specific checklist items.

Pillar	ShopFlow Implementation	Status
Operational Excellence	Container Insights enabled; deployment circuit breaker; ECS Exec for debugging; CloudWatch Container Insights JSON logging to CloudTrail	Complete
Security	Task-level IAM roles (not shared); secrets via Secrets Manager injection; no public endpoints; security groups per service	Complete
Reliability	Multi-AZ task placement (1+ task per AZ at min capacity); ALB health checks; deployment circuit breaker auto-rollback	Complete
Performance Efficiency	ARM64 (Graviton) for 20% cost/perf improvement; auto scaling on CPU + request count; Fargate Spot for non-critical	Complete
Cost Optimization	Fargate Spot for stateless services (50% savings); ARM64 vs x86 (20% savings); Savings Plans for EC2 min capacity	Savings Plans pending

TIP ECS Compute Savings Plans

AWS Compute Savings Plans give up to 50% discount on Fargate in exchange for a 1- or 3-year commitment to a minimum hourly spend. ShopFlow baseline: 15 tasks x \$0.20/hr = \$3/hr minimum. A 1-year Savings Plan at \$3/hr gives 36% discount on all Fargate usage up to that hourly rate. Savings Plans automatically apply to both Fargate and Lambda usage. Recommended after 3+ months in production when baseline task count is stable.
